

Agentic AI for Economic Research

Session 2: live workflows, case studies, and lessons learned

Marco Le Moglie

Catholic University of the Sacred Heart

Agentic Economic Research seminar | Session 2

Mini-course for PhD students and faculty | hours 3–4

Slides source on GitHub

What we see today

1. Agent interfaces live screens

Chat, desktop apps, IDE, CLI — menus and options in real tools.

2. Success stories projects at equilibrium

Selected projects where the agent workflow is clean and repeatable.

3. Second brain Obsidian vault & memory

How it works; how I organise mine; how you might organise yours.

4. Work in progress live screens

Bootstrap, supervision, errors — all on real screens.

Part 1 – Agent interfaces

Part 1: four interface families, live

Today we open the real tools

- Chat window
- Desktop agent apps
- IDE agents
- CLI in the terminal

Live demo goal

Not a feature tour for its own sake: show *where* autonomy lives in each interface — and which menu choices matter for research.

Part 2 – Success stories

Part 2: projects at equilibrium

For each case: objective, current workflow, what we learned, problems hit.

- **YouTube channel** — economics explainer pipeline.
- **Newsletter** — ASSIST Weekly harvest-to-send stack.
- **Mafia culture** — datasets, reports, documentation agents.
- **Paper review** — referee-report skills and adversarial checks.
- **Purge** — journal-target skill adapted to a new outlet.

Common thread: repeatable folder discipline + human gates.

Success story: Cosa fanno gli Economisti

Live links: [YouTube](#) | [Instagram](#) | [Facebook](#)

Objective

- Turn economics papers into short explainer videos for a general audience.
- Automate production, SEO, YouTube programming, and cross-posting (FB/IG).

Current state & workflow

- End-to-end: paper → cover → NotebookLM video → cleanup → multilingual upload → Buffer social.
- Three layers: *Directives* (SOP) → orchestration → deterministic execution.
- Lanes: Enea (production), Romolo (YouTube/SEO), Marcello (social), Mercurio (backup), Ulisse (scouting).

What I learned

- **Connectors:** MCP (NotebookLM), APIs (YouTube, Buffer), CLI (nlm, ./workflow).
- **Structured rules + multi-agent** workflows; truth in `video_tracking.json`.
- **Token control:** targeted git push, recover partial uploads, no blind re-runs.
- **YouTube SEO:** titles, playlists, competitor comments, monthly report.
- **Tool drift:** Antigravity/Codex upgrades broke rules, MCP paths, terminal defaults — re-audit SOPs after each version jump.

Success story: *ASSIST Weekly*

Live link: assistweekly.org **Email example:** *Issue 2026-07-06*

Objective

- Weekly brief: datasets, tools/methods, working papers.
- Stack: discover → QA → email → landing/signup.

Current state

- #13 sent 2026-07-06; 2026-07-13 shortlist ready.
- Brevo campaigns; Cloudflare Pages/Worker; internal dashboard.

Three automations

- **Wed** harvest → shortlist + dashboard
- **Thu** render after `APPROVED` .flag
- **Mon** Brevo send after `EMAIL_APPROVED`

Manual gates: dashboard review → flags.

What I learned

- Brevo; Cloudflare deploy; editorial dashboard.
- Scraping/APIs (GitHub, Dataverse, NBER, CEPR).
- Brand/trademark; anti-slop frontend; LaunchAgents.

Success story: Skill /referee-report

Objective: turn an annotated manuscript into a constructive English referee report in Marco's voice, with adversarial checks and style archive.

Skill /referaggio — main steps

- 1 **Intake** — Marco uploads annotated PDF to reviews/.
- 2 **Scaffold** — folder {MS-ID}-{JOURNAL}/ + checklist.
- 3 **Comments** — extract, expand, organize; drop nonsense *with reason*.
- 4 **Econoclast** — adversarial pass; **manual PDF verify** each finding.
- 5 **Plan** — proposed_points.md; **STOP: Marco approves**.
- 6 **Draft** — referee_report.tex/md/pdf (conciliatory tone).
- 7 **Anti-slop** — prose-check on draft.
- 8 **Archive** — style example; **STOP: final Marco approval**.

Human gates (non-negotiable)

Wait for PDF upload (step 1). Clarify ambiguous comments (step 3). Approve integrated plan before drafting (step 5). Approve final report before archive (step 8). Econoclast supplements — does not replace Marco's judgment.

Success story: Skill /journal-style-paper-rewrite

Objective: adapt a frozen paper (e.g. J. Popul. Econ. → Demography) — new framing and prose, *same* empirical package.

Skill workflow — main parts

- 1 **Lock source** — read memory; **do not edit frozen paper**.
- 2 **Preserve empirics** — no new regressions/tables unless Marco reopens.
- 3 **Design prompt** — **ask Marco if missing** (journal, framing, tone).
- 4 **Target-journal plan** — title/abstract, intro, results rhetoric.
- 5 **Anti-slop** — prose-check on new version.
- 6 **Two outputs** — AA_paper_<journal>.tex + matching report.
- 7 **Color logic** — mark only text actually changed vs. source.
- 8 **Compile & verify** — source untouched; report matches new version.

Human gates

Frozen paper stays read-only. Journal-design prompt required before any rewrite. Marco approves framing; agents do not improvise empirical reopening. Color only real edits

Part 3 – Second brain

Part 3: memory and the Obsidian second brain

- Session 1 introduced *why* a curated Markdown vault beats blind RAG for research traceability.
- Today: **how** it works in practice — my vault layout

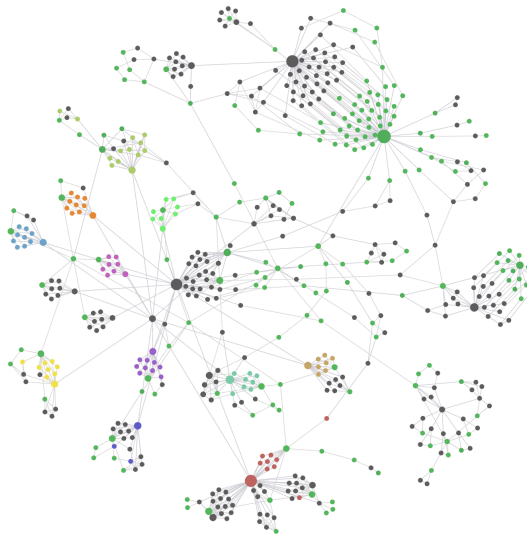
We will show

- Folder organization: General, Areas, projects, wiki vs. raw notes.
- Brain bootstrap: opening and closing a session
- Tools, plugins, protocols, skills

Takeaway for you

- Start small: one project folder, eight canonical files.
- Write decisions in Markdown, not only in chat.
- Treat the vault as the restart point every session.

Part 3: My Second Brain



Part 3: how I built and wired the Brain

Build & folder structure

- Obsidian vault = **Markdown source of truth** (not blind RAG).
- 01_GENERAL/ — agent protocols, skills, integrations, scripts.
- 02_AREAS/ — Teaching, Research, Other; one folder per project.
- Per project: 8 canonical files + wiki/ + raw/chats/.
- Populate via `create_project.py`, session closeout, LLM Wiki ingest.

Integrations (support layers)

- **Anti-sloppiness** — Slopless on formal prose.
- **Headroom** — context compression (tokens, long logs).
- **Memanto** — typed semantic recall;
- **code-review-graph** — impact-aware code review.
- **Econoclast** — adversarial referee on empirical papers.
- **Style / taste** — voice profile + archived examples.

Protocols & skills

- **LLM Wiki** — promote stable notes into project wiki/.
- **Caveman** — terse agent mode; saves tokens, keeps substance.

“Open brain session”

- Read global + project memory (INDEX, CURRENT_STATE, ...).
- Inspect live files; summarize state + plan — *before* any edits.

“Close brain session”

- Update CURRENT_STATE, NEXT_ACTIONS, DECISIONS, LOG.
- Promote durable facts to AGENT_MEMORY / wiki/; optional Memanto sync.

Part 4 – Work in progress

Part 4: WIP — setup, supervision, and fixes

1. Project bootstrap

- My standard start: **Brain** → **GitHub** repo → **Overleaf** sync.
- **Example:** *Mafia migration* — matched DiD pipeline, analysis plan, wiki, push to GitHub.

2. When the machine needs a human gate

- **Purge** — 1940 INE profession proxies: terrible embedded OCR; rebuilt with image re-OCR, column/row templates, census caps; I spot-checked flagged cells.
- Pattern: agent proposes fix → I verify against source → we lock the auditable pipeline.

3. Interactive contrast

- Build the same table with **plain ChatGPT** vs. a **Codex coding agent** on real project files.

Live demos may fail — that is part of the lesson.